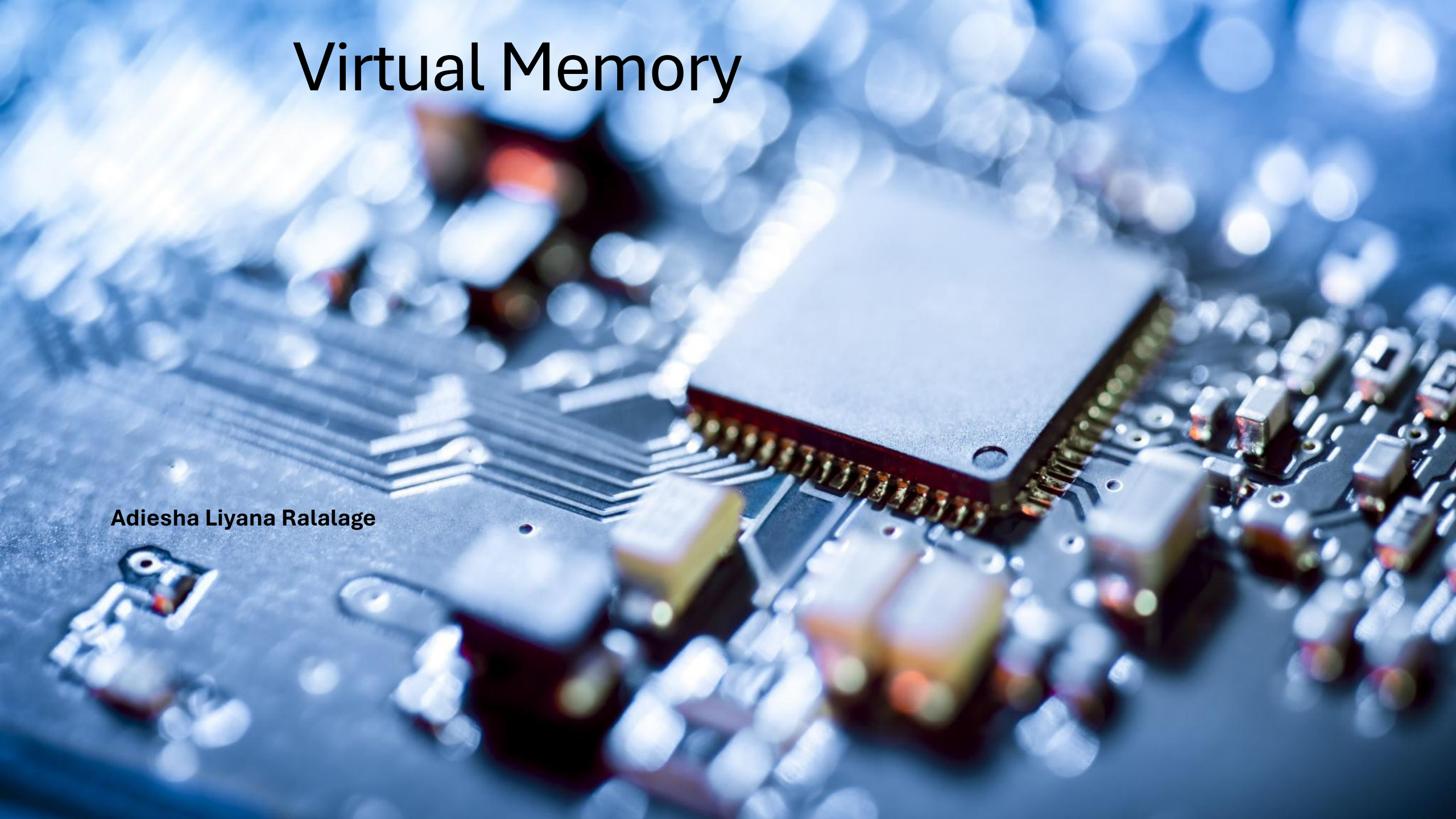


Virtual Memory

Adiesha Liyana Ralalage



Virtual memory	Uses secondary storage (e.g., disk) as if it were main memory (RAM). Programs use their own addresses, which the system automatically translates to real physical locations. Its size is capped by the system's addressing scheme and available secondary storage — not by the actual amount of RAM.
Virtual address	An address in virtual memory, used as if it pointed to main memory.
Virtual address space	The virtual memory assigned to a single process.
Address space	The full range of addresses a process can use.
Real address	The address of an actual location in main memory (RAM).

Hardware and Control structures

- There are two fundamental breakthroughs that emerged through paging and segmentation.
 1. All memory references are logical addresses that are dynamically translated into physical addresses at run time.
 2. A process may be broken up into a number of pieces that don't need to be contiguously located in main memory during execution.
- If these two characteristics are present:
 - It is not necessary that all the pages or segments to be in the main memory during the execution of the program.

Execution of a process

- Operating system brings into main memory a few pieces of the program
 - Resident set
 - Portion of process that is in main memory
- An interrupt is generated when an address is needed that is not in main memory. (when paging is used, this is called page fault).
- Operating system places the process in a blocking state.

Execution of a process

- Piece of process that contains the logical address is brought into main memory
 - Operating system issues a disk I/O Read request
 - Another process is dispatched to run while the disk I/O takes place
 - An interrupt is issued when disk I/O is complete, which causes the operating system to place the affected process in the Ready state.

What are the implications of this strategy?

- More processes may be maintained in main memory.
 - Only parts of processes are loaded into the memory, therefore more room for new processes.
 - More ready state processes can be stored in the main memory.
- Processes that are larger than the main memory can be run.
 - Previously, programmer had to write overlays to load only part of the program to the main memory.
 - But now, with virtual memory based on paging or segmentation, this task is left to the OS and the hardware.

Real vs Virtual Memory

Real memory

- Main memory, the actual RAM

Virtual memory

- Memory perceived by the process
- Allows for effective multiprogramming and relieves the user of tight constraints of main memory

Paging VS Segmentation

How memory is divided and addressed — whether it's **virtual** is a separate choice, shown below.

Paging

uniform · fixed size



Memory pre-cut into equal **frames**.
A page fills one frame; space
left inside is wasted —
internal fragmentation.

- Split into fixed-size frames
- Pages made by the compiler / OS
- Internal fragmentation only
- Page table maps page → frame
- OS tracks a free-frame list
- Address = `page # + offset`

Segmentation

variable · logical size



Whole **segments** of varying
size placed where they fit —
gaps between them wasted:
external fragmentation.

- Memory not pre-partitioned
- Segments defined by the programmer
- External fragmentation only
- Segment table: base + length
- OS tracks a list of free holes
- Address = `segment # + offset`

+ VIRTUAL MEMORY

An orthogonal layer — it applies to **either** scheme above.

The only thing it changes: whether the whole program must be in RAM before it can run.

SIMPLE — no virtual memory

Every page or segment must be in RAM to run, unless
overlays are used.

VIRTUAL

Only the needed pages/segments load; the rest stay on disk,
fetched on demand. Loading one in may write another out.

When Memory Is Almost Full

- Consider the following scenario:
 1. Main memory is almost full.
 2. A process needs a page that is **not currently in main memory**.
 3. The OS must:
 - **Swap out** a page from main memory to secondary storage.
 - **Bring in** the requested page.
 4. Suppose the page that was just swapped out is needed **almost immediately**.
 5. The OS must:
 - Bring that page back into main memory.
 - Swap out another page to make room.
 6. If this happens repeatedly, the OS spends most of its time **swapping pages in and out** rather than executing processes.

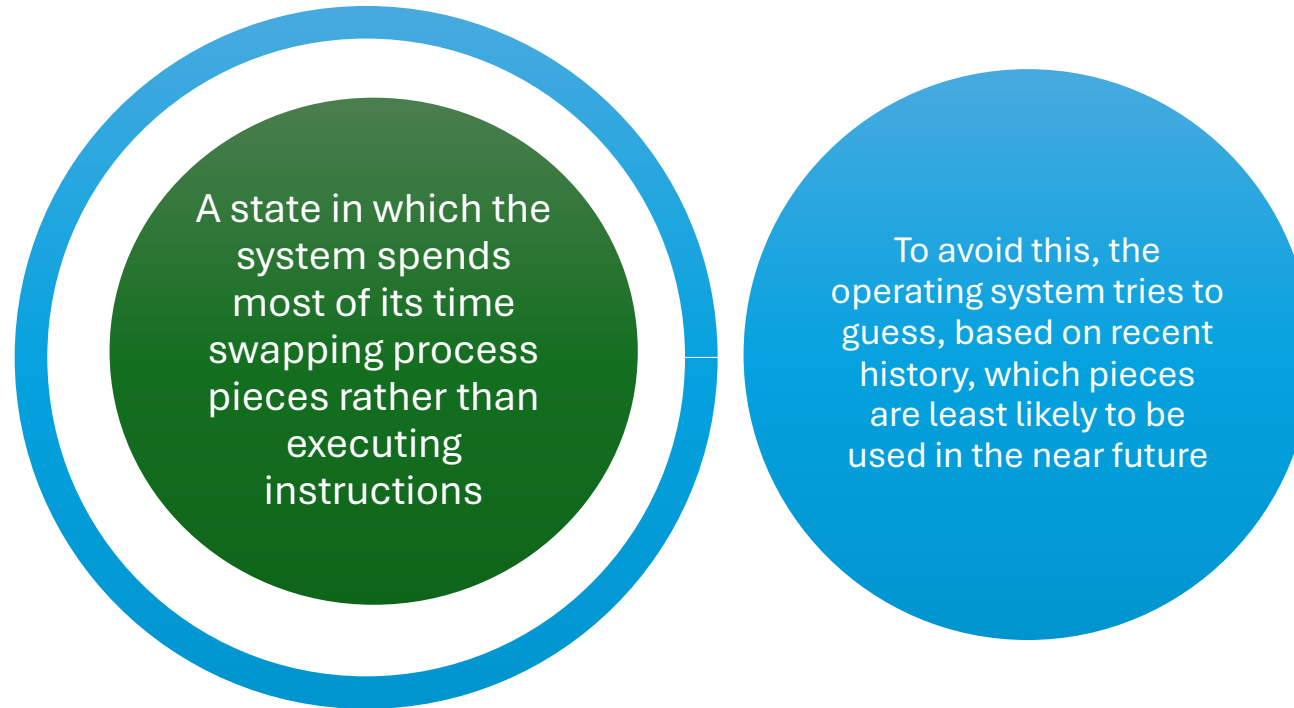
Too much time managing memory → too little time executing processes

Thrashing Animation



thrashing.html

Thrashing



How Can We Avoid Thrashing?

- Use the principle of locality.
 - A program's memory references tend to **cluster in a relatively small number of pages** during a given period of time.
 - Therefore, during a short period, a process typically needs only a **small subset of its pages**.
 - The OS can use this behavior to make better decisions about **which pages to keep in memory and which pages to evict**.
 - If the pages that a process is actively using remain in memory, the number of **page faults and page swaps** can be reduced.
 - This helps prevent the system from spending most of its time **paging instead of executing processes**.

What do we need for virtual memory scheme to work?

- For virtual memory scheme to be practical, we need:
 1. Hardware support for paging or segmentation or both.
 2. Operating system must include software for managing the movement of pages and/or segments between secondary memory and main memory

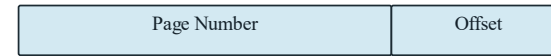
Control structures necessary for virtual memory

- Now, we will look at necessary control structures that are created and maintained by the OS but are used by the memory management hardware.
- Usually, when we talk about virtual memory, they are associated with systems that use paging.
 - However, you can use segmentation or combination of paging with segmentation for virtual memory.
- Each process has its own page table—This is the control structure.

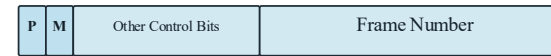
Control structures necessary for virtual memory

- Each process has its own page table (The control structure)
 - Each **page table entry** (PTE) contains the frame number of the corresponding page in main memory. (we already talked about this when we looked at simple paging)
 - The page table is needed for virtual memory with little bit more information on it.
 - We looked at the page table in the last lecture, let's try to analyze the structure of the page table bit more.

Virtual Address



Page Table Entry

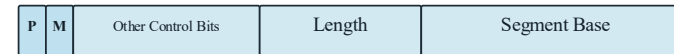


(a) Paging only

Virtual Address



Segment Table Entry

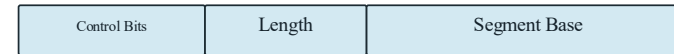


(b) Segmentation only

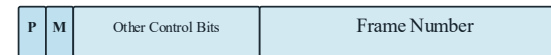
Virtual Address



Segment Table Entry



Page Table Entry



P = present bit
M = Modified bit

(c) Combined segmentation and paging

Figure 8.1 Typical Memory Management Formats

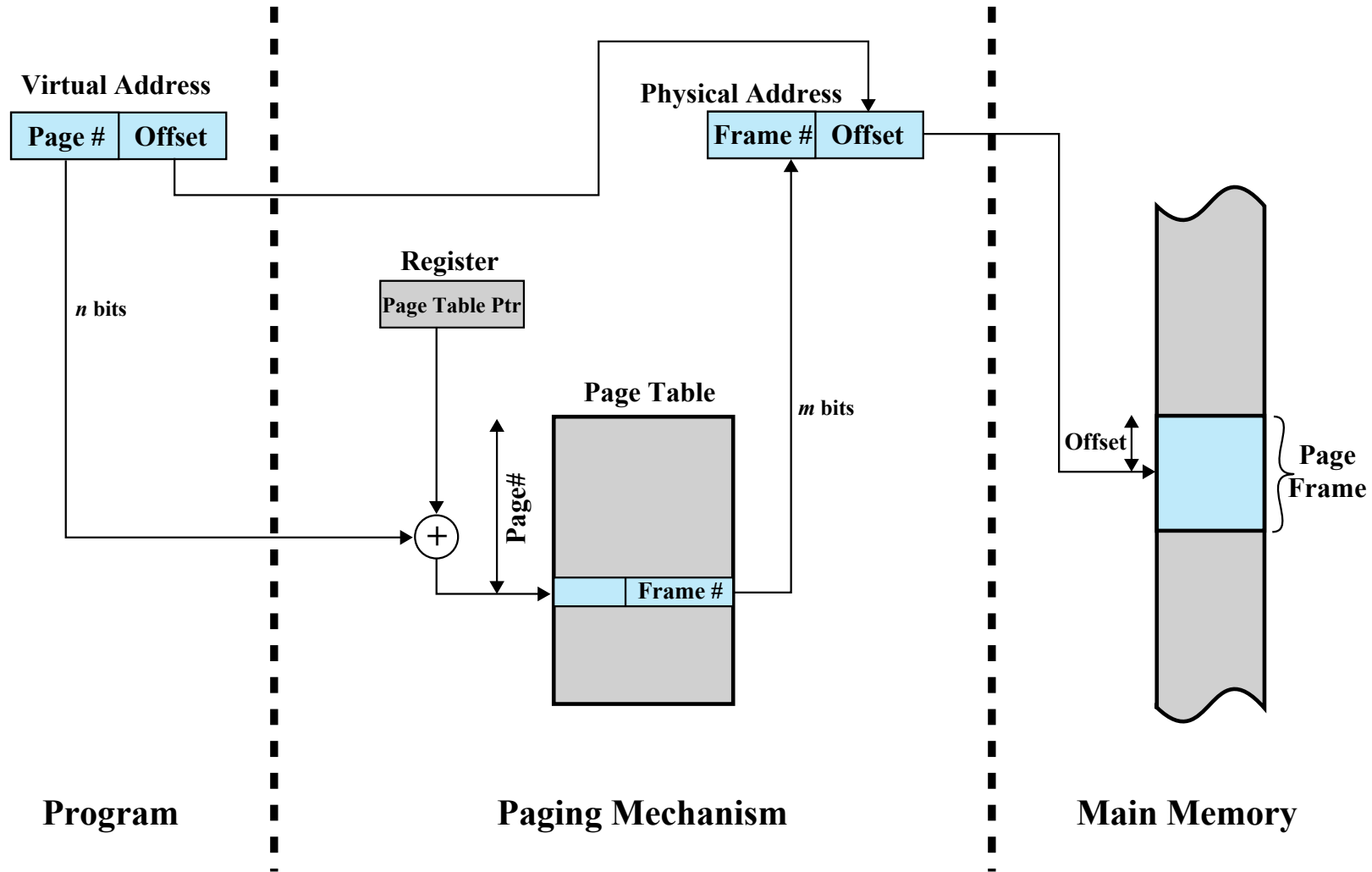


Figure 8.2 Address Translation in a Paging System

Question

- Suppose Process A wants to change the page table entry for one of its virtual pages so that the page maps to a different physical frame. Can Process A simply modify the page table entry itself? Explain why or why not.
 - **No. Process A cannot directly modify its page table entry.** The page table is managed by the operating system and is protected from user processes. If a process needs to change its memory mapping, it must request the operating system to do so through a **system call**. The OS then validates the request and, if permitted, modifies the page table.

Address translation in a paging system animation



paging-address-translation.html

How large can a page table be?

- Let's work through an example.
- Assume:
 - Address size = **32 bits**
 - Byte-level addressing (1 byte per addressable unit)
 - Page size = **4 KB = 2^{12} bytes**
 - Each page-table entry (PTE) = **4 bytes**
- Step 1: What is the maximum virtual address space?
 - A 32-bit address can represent 2^{32} different addresses.
 - Since we have byte-level addressing:
 - $2^{32} \text{ bytes} = 4\text{GB}$
 - Therefore, the maximum virtual address space is **4 GB**.
- Step 2: How many pages can a process have?
 - **Each page contains 2^{12} bytes**
 - $\frac{2^{32}\text{bytes}}{2^{12}\text{bytes/page}} = 2^{20}\text{pages}$
 - **A process can have at most 2^{20} pages.**

How large can a page table be?

- Assume:
 - Address size = **32 bits**
 - Byte-level addressing (1 byte per addressable unit)
 - Page size = **4 KB = 2^{12} bytes**
 - Each page-table entry (PTE) = **4 bytes**
- **Step 3: How large can a page table be?**
 - There is one page-table entry for each page.
 - Each entry is 4 bytes.
 - $2^{20} \text{ entries} \times 4 \text{ bytes per entry} = 2^{22} \text{ bytes} = 4 \text{ MB}$
 - Therefore, the maximum page-table size for one process is 4MB
- Now consider a system with many processes:
 - If 100 processes are running, and each has a 4 MB page table:
 - $100 \text{ MB} \times 4 \text{ MB} = 400 \text{ MB}$

How large can a page table be?

- **Step 3: How large can a page table be?**

- There is one page-table entry for each page.
- Each entry is 4 bytes.

- $2^{20} \text{ entries} \times 4 \text{ bytes per entry} = 2^{22} \text{ bytes} = 4 \text{ MB}$

- Therefore, the maximum page-table size for one process is 4MB

- Now consider a system with many processes:

- If 100 processes are running, and each has a 4 MB page table:
 - $100 \text{ MB} \times 4 \text{ MB} = 400 \text{ MB}$

- In this example, every process needs a page table of 4MB.

- **Can we afford to dedicate this much memory to page tables?**

- **How can modern operating systems reduce the memory required for page tables?**

Paging

- Large chunk of memory is devoted to page tables.
- What can we do to improve this?
 - What if we keep the page tables in the virtual memory as well?
- How can we keep track of the paging of the page tables?
 - Hierarchical page table structure

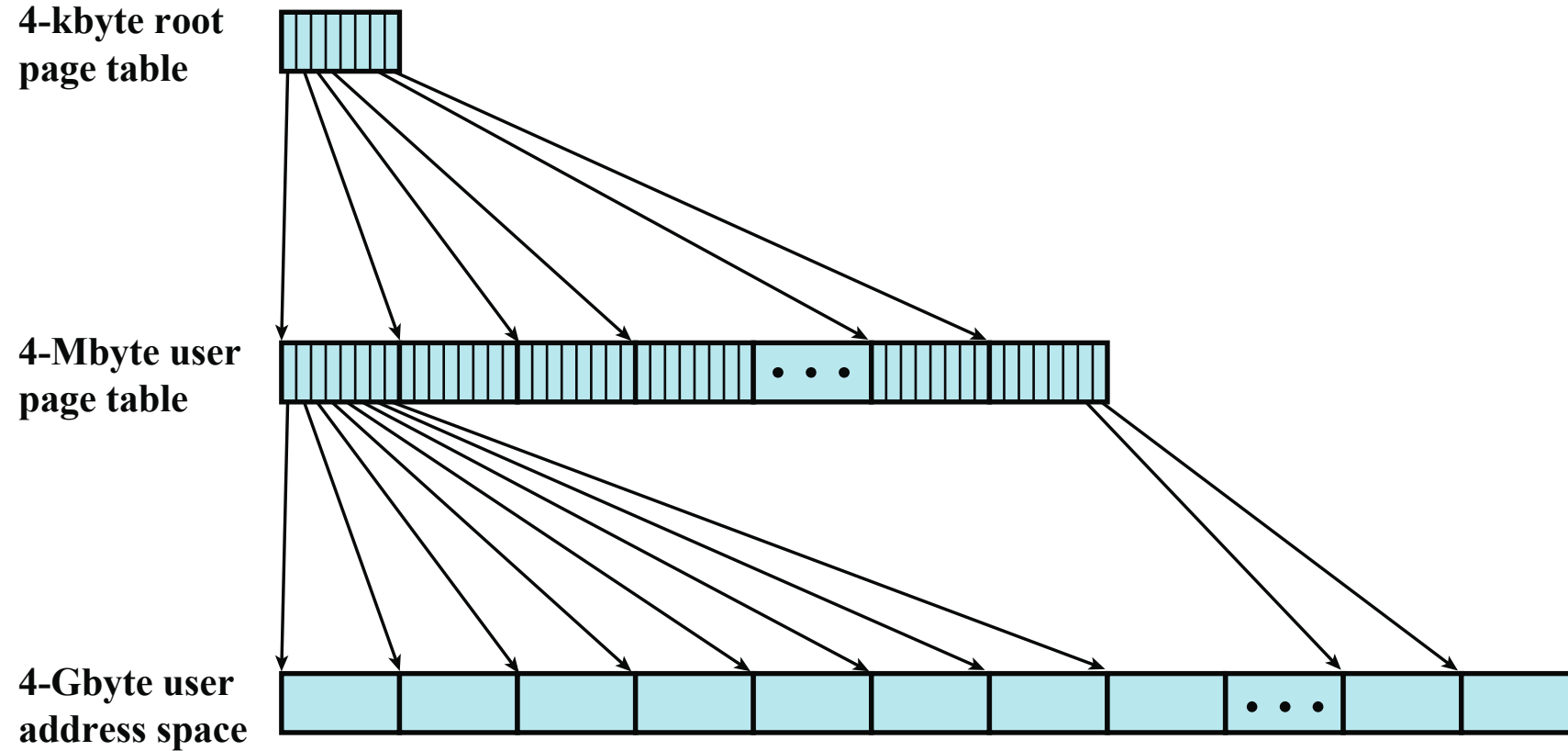


Figure 8.3 A Two-Level Hierarchical Page Table

Paging

- Remember root page table always exists in the main memory.

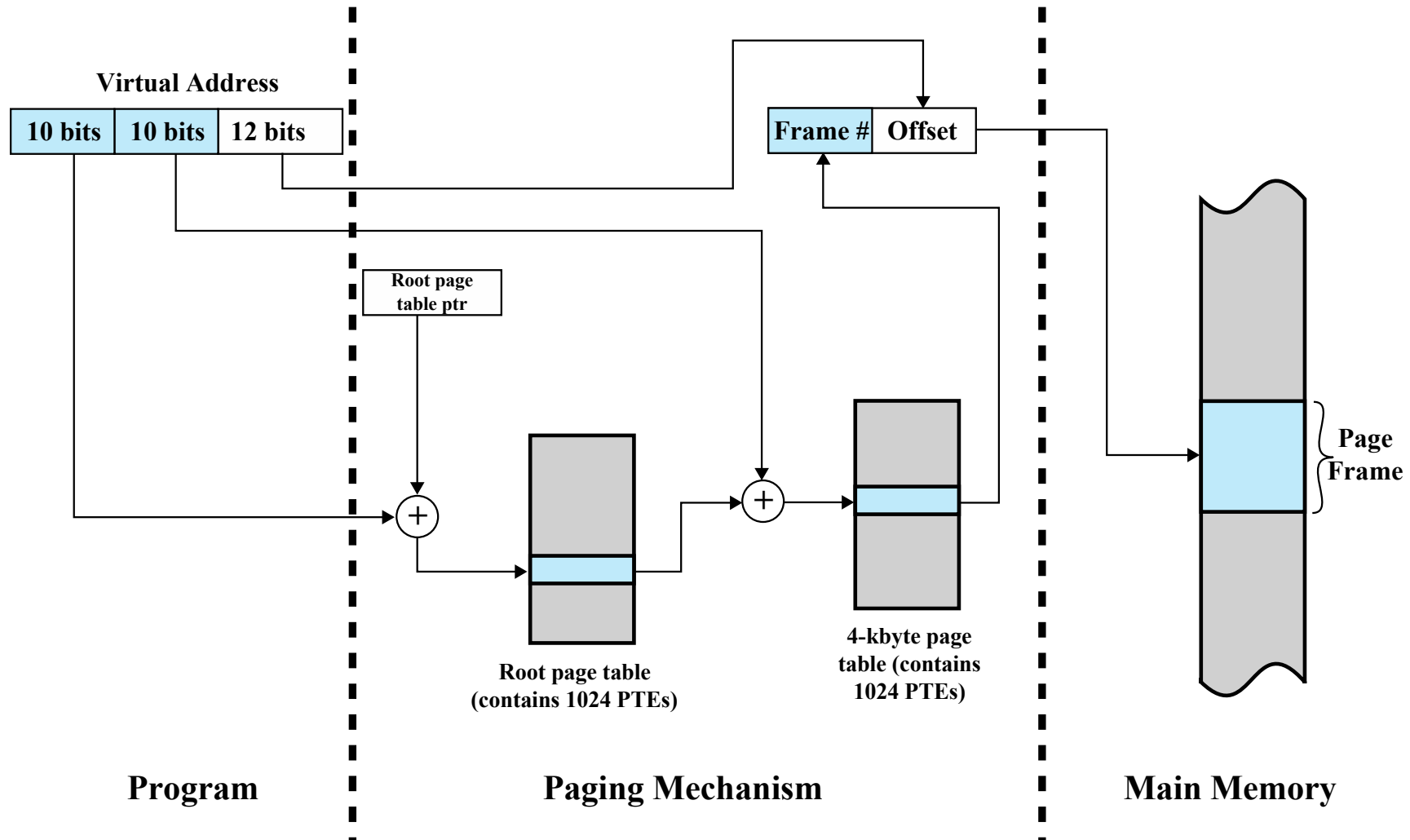


Figure 8.4 Address Translation in a Two-Level Paging System

Paging

- Modern chips have the ability to keep 4-5 levels of page table hierarchy.

Inverted Page Table

- A key drawback of the conventional page tables discussed so far is that their size is proportional to the virtual address space — and is paid per process — regardless of how much memory the process actually uses.
- An alternative approach is the **Inverted Page Table**.
- Instead of keeping one entry per virtual page per process, it keeps a single system-wide table with one entry per physical frame, recording which page of which process currently occupies that frame. Its size is therefore bounded by physical memory, not by the virtual address space.
- Since the table is organized by frame rather than by virtual page number, we can no longer index it directly.
 - A hash function on the (process ID, virtual page number) pair locates the matching entry.

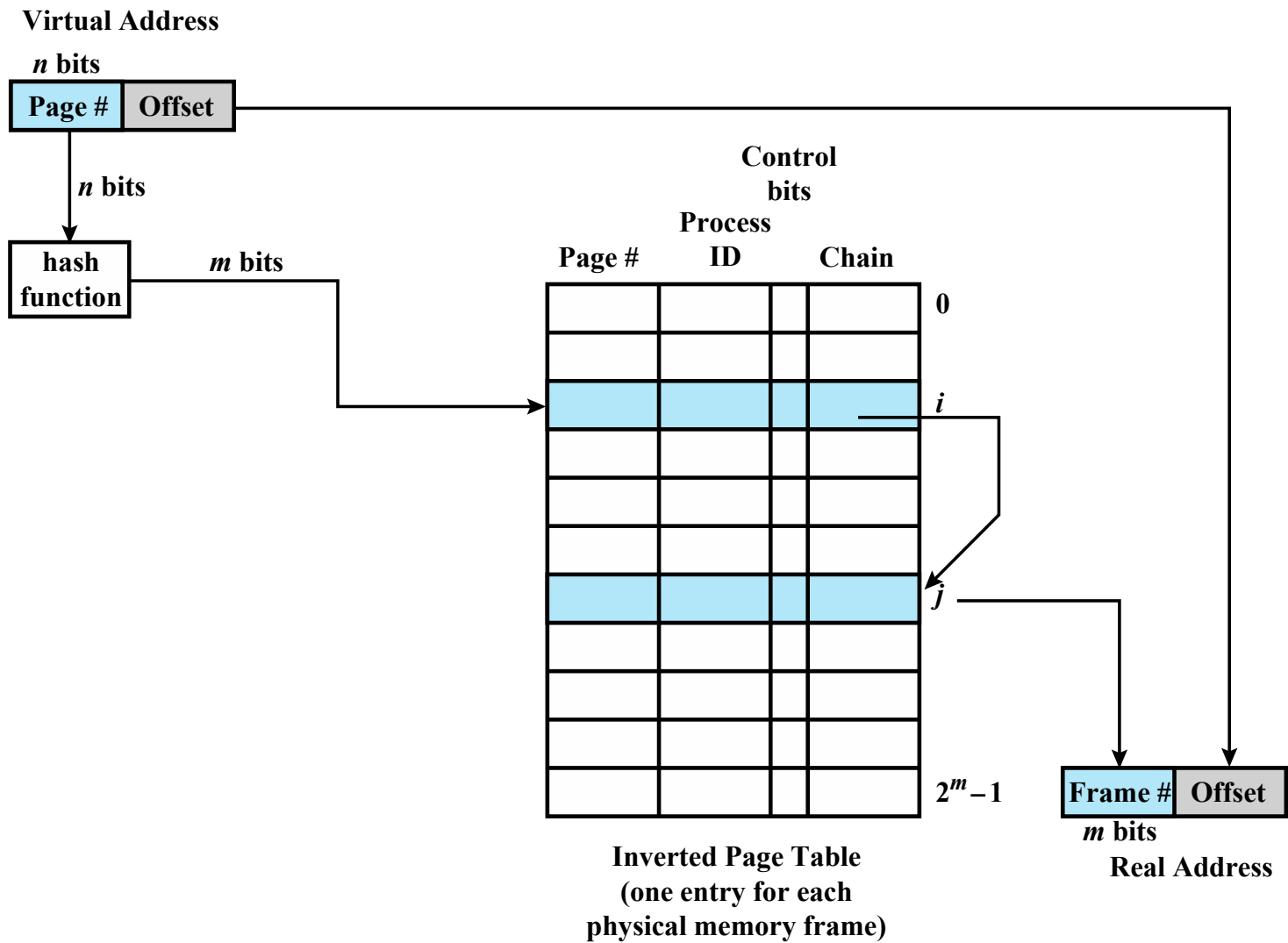


Figure 8.5 Inverted Page Table Structure

Inverted page table

- What are the implication?
 - Fixed proportion of real memory is required for the tables regardless of the number of processes or virtual pages supported.
 - One page table for all processes since the entries are keeping track of frames instead of pages of a process.
 - Is this better?
- Structure is called inverted because it indexes page table entries by frame number rather than by virtual page number.

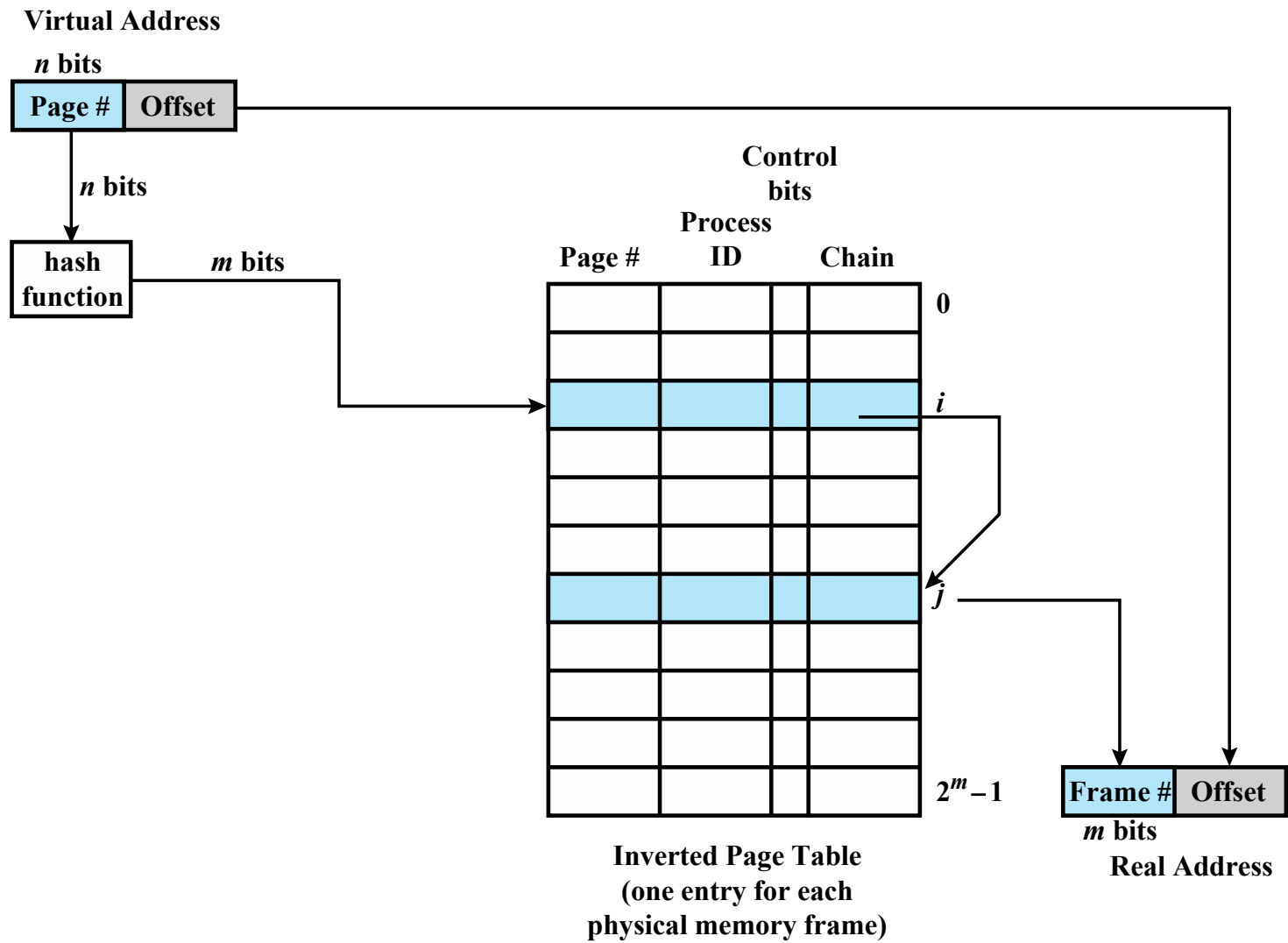


Figure 8.5 Inverted Page Table Structure

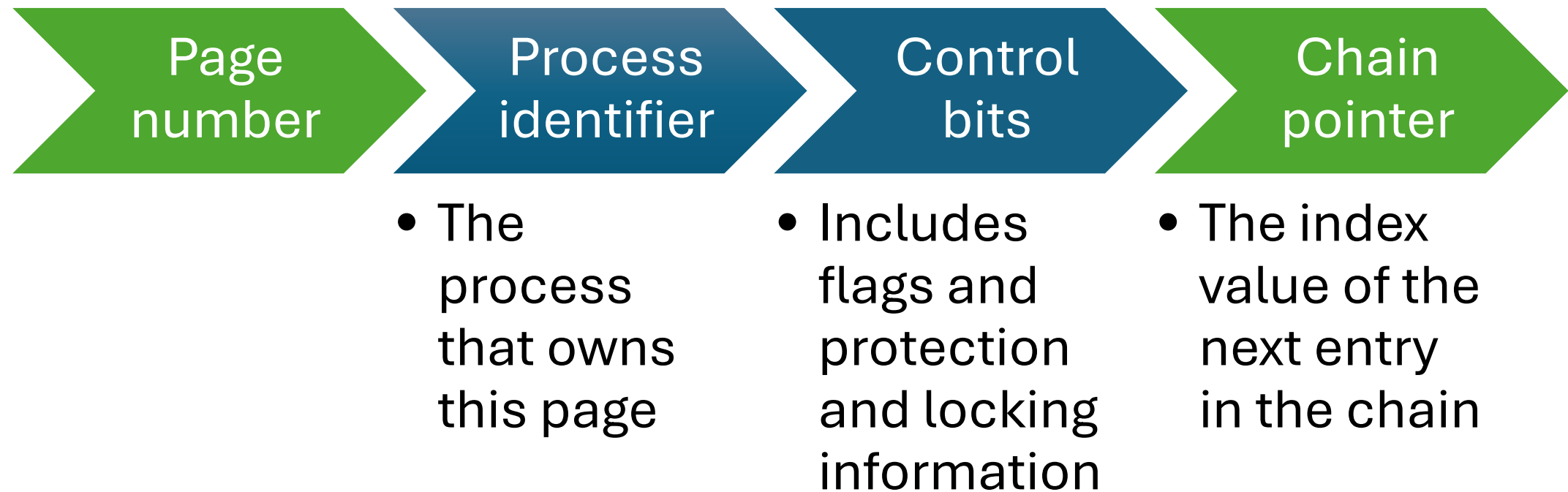
Animation



inverted-page-table.html

Inverted Page Table

- Each entry in the page table contains the following information:



- Modern systems use multi-level page tables instead of inverted page tables.
- Yet, they're the historically interesting alternative more than the common one today.

Translation Lookaside Buffer (TLB)

- Question:
 - How many memory accesses are needed to access a particular memory address using this paging scheme in the best case?
 - You first need to access the page table to locate the frame address, then use it to form the physical address, which allows access to the intended memory location.
 - If there is page fault, then we need to bring the data from the secondary storage as well.
 - Basically, this paging scheme has the effect of doubling the memory access time.
- Due to this reason, most virtual memory access schemes use a special high-speed cache for page tables called **Translation Lookaside Buffer**.

Translation Lookaside Buffer (TLB)

- Due to this reason, most virtual memory access schemes uses a special high-speed cache for page tables called **Translation Lookaside Buffer**.

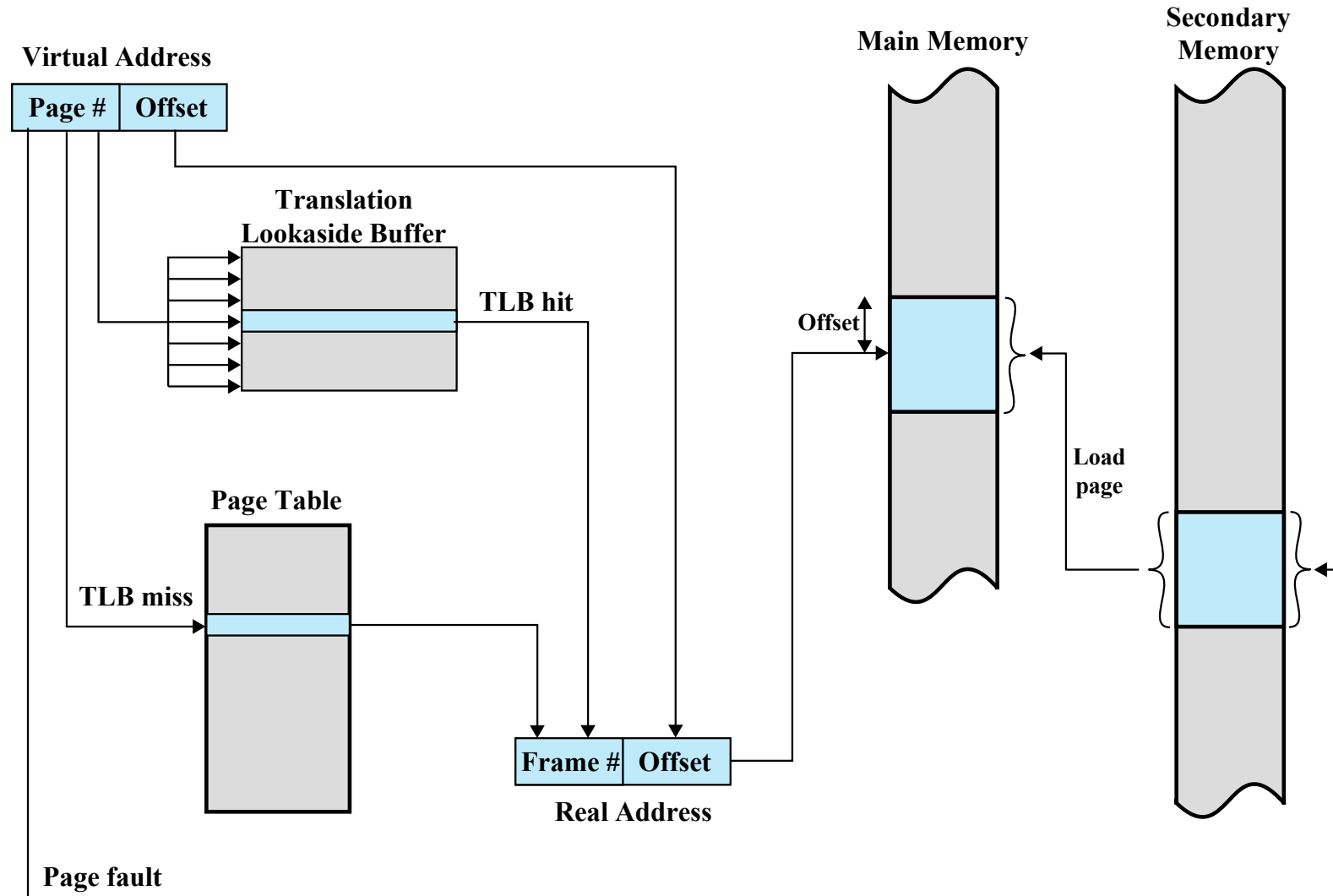


Figure 8.6 Use of a Translation Lookaside Buffer

Translation Lookaside Buffer (TLB)

- Each entry in the TLB contains page number as well as the complete page table entry.
 - Some CPU architectures allow pages of multiple processes to be in the TLB at the same time—in fact most modern CPUs allow this. **(ASID is needed)**
 - In older systems, TLB only contains page entries of the currently running process, and after each context switch the TLB is flushed and loaded with new pages of the currently running process.
- Regardless, CPU still needs to go through the entries of the TLB to find an entry, but this is still costly, even the access time is low.
 - Thus, what hardware manufacturers did was to introduce circuitry to search all the TLB entries at once.
 - This is called **associative mapping**.
 - This introduce complexity to the circuitry and becomes more expensive to build—but we keep size of the TLB small reducing the overall cost.

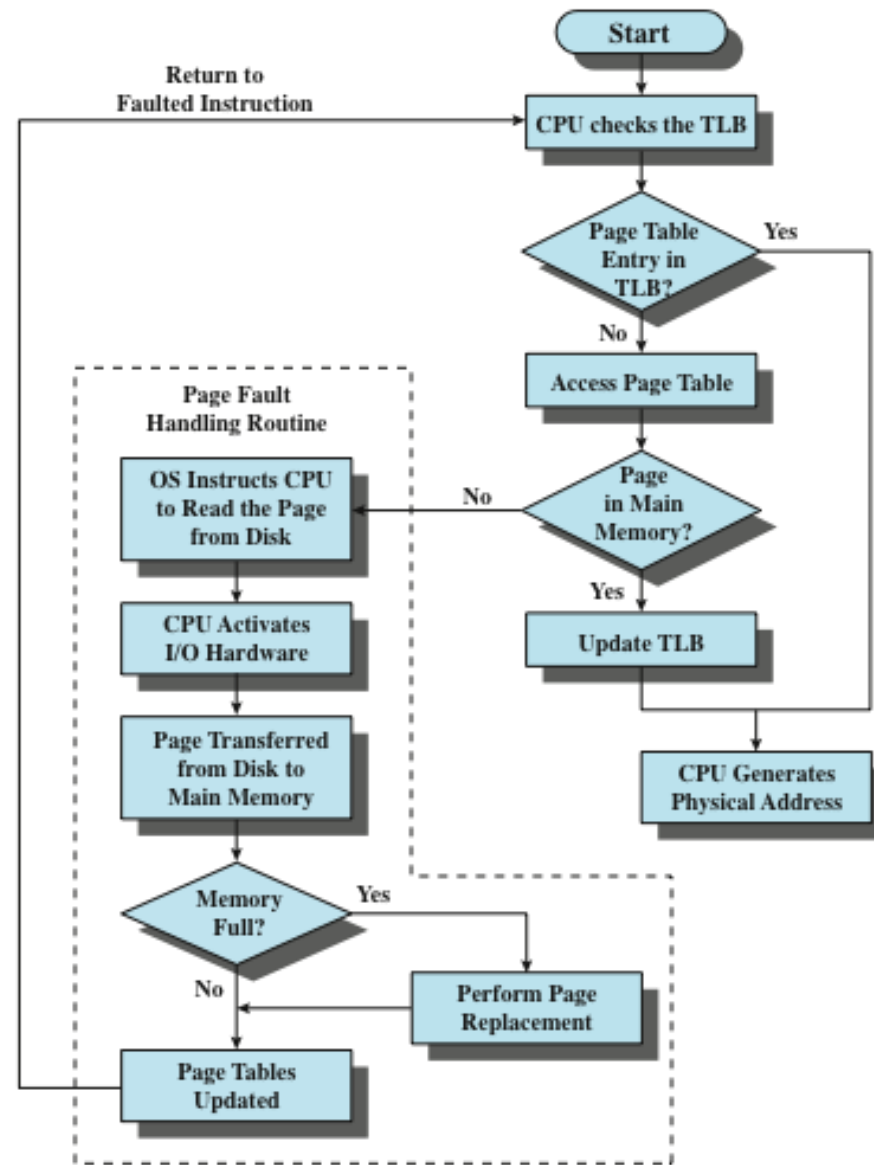


Figure 8.7 Operation of Paging and Translation Lookaside Buffer (TLB) [FURH87]

Animation



tlb-operation.html

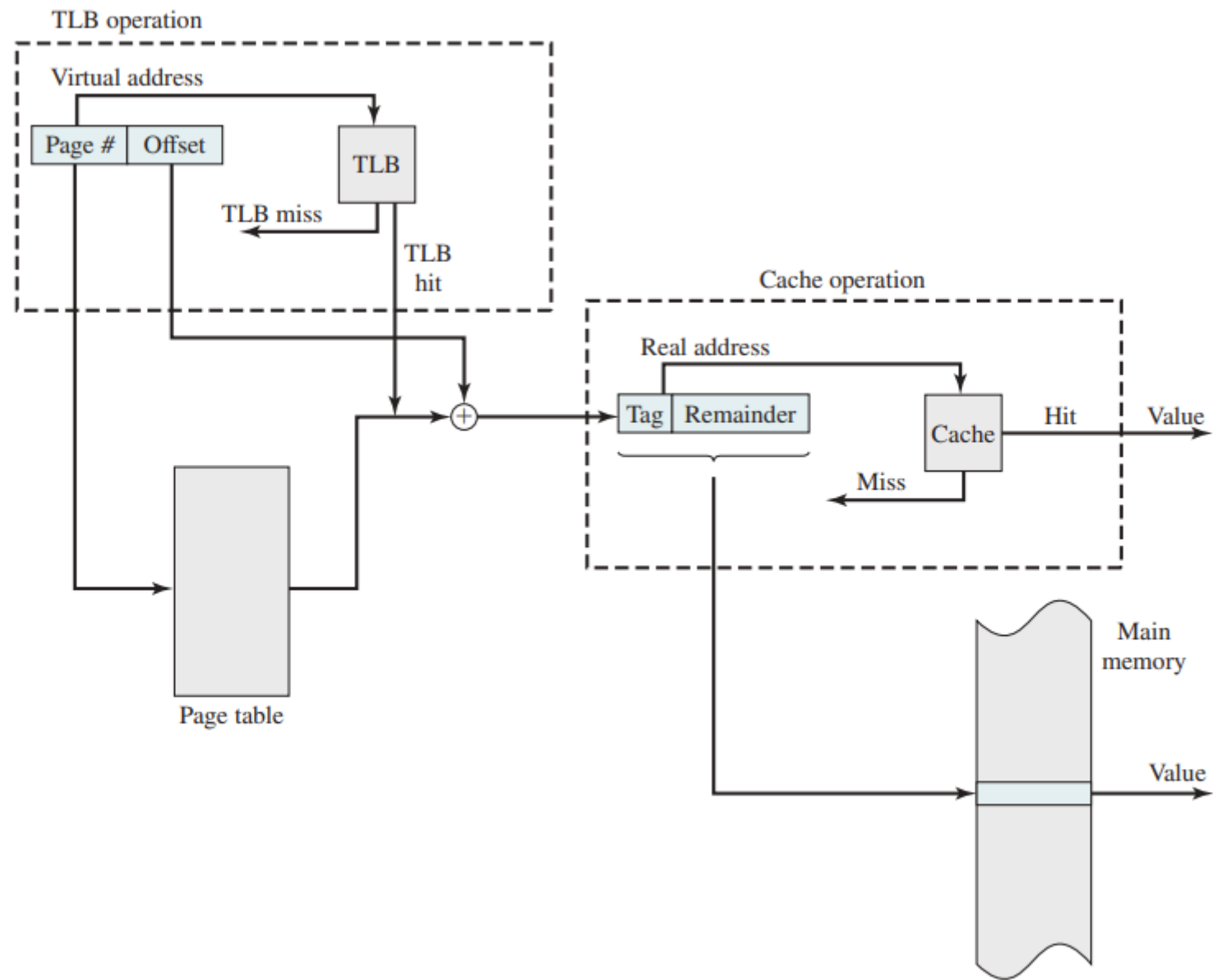


Figure 8.9 Translation Lookaside Buffer and Cache Operation

Animation

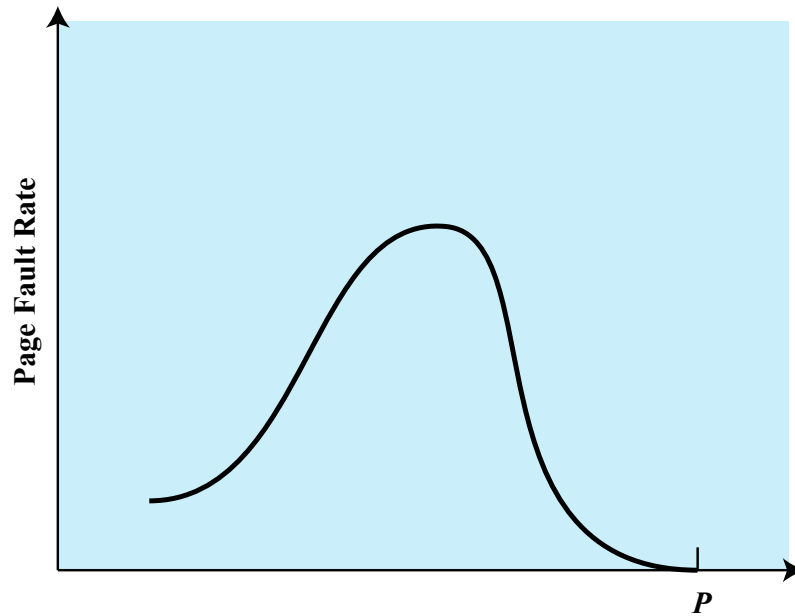


tlb-cache-operation.html

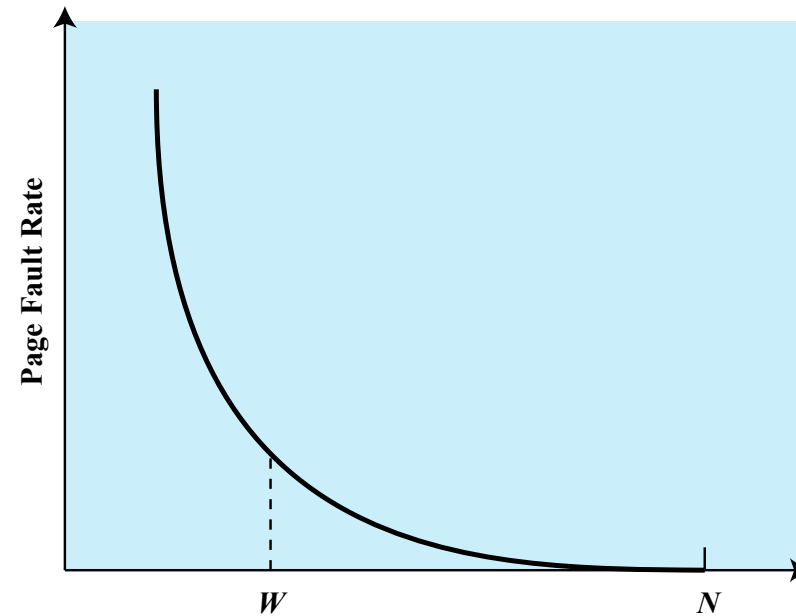
Page size

- There are two major factors to consider.
 1. Internal fragmentation
 - Smaller page sizes will lead to less internal fragmentation.
 - But will result in larger page table size.
 - Could occur double page faults.
 - First to bring in the needed portion of the page table, and second to bring in the process page.
 2. Physical characteristics of the secondary storage.
 - Larger page sizes are better for secondary storages which are rotational as it would result in much efficient data transfer.
 3. Size of the main memory and program size
 - Object oriented techniques may decrease the locality of references as it encourage to use a lot of smaller programs that is scattered across the memory.

Paging behavior



(a) Page Size



(b) Number of Page Frames Allocated

P = size of entire process

W = working set size

N = total number of pages in process

Figure 8.10 Typical Paging Behavior of a Program

Segmentation

- Implications:
 - Segmentation allows the programmer to view memory as consisting of multiple address spaces or segments
 - Instead of treating memory as a single, continuous block, segmentation splits it into smaller, logically distinct regions (segments). Each segment has its own starting address and size, and the programmer or operating system can treat these segments independently

Segmentation

- Advantages:
 - Simplifies handling of growing data structures
 - Allows programs to be altered and recompiled independently
 - Lends itself to sharing data among processes
 - Lends itself to protection

Segment table

- Each segment table entry contains the starting address of the corresponding segment in main memory and the length of the segment
- A bit is needed to determine if the segment is already in main memory
- Another bit is needed to determine if the segment has been modified since it was loaded in main memory

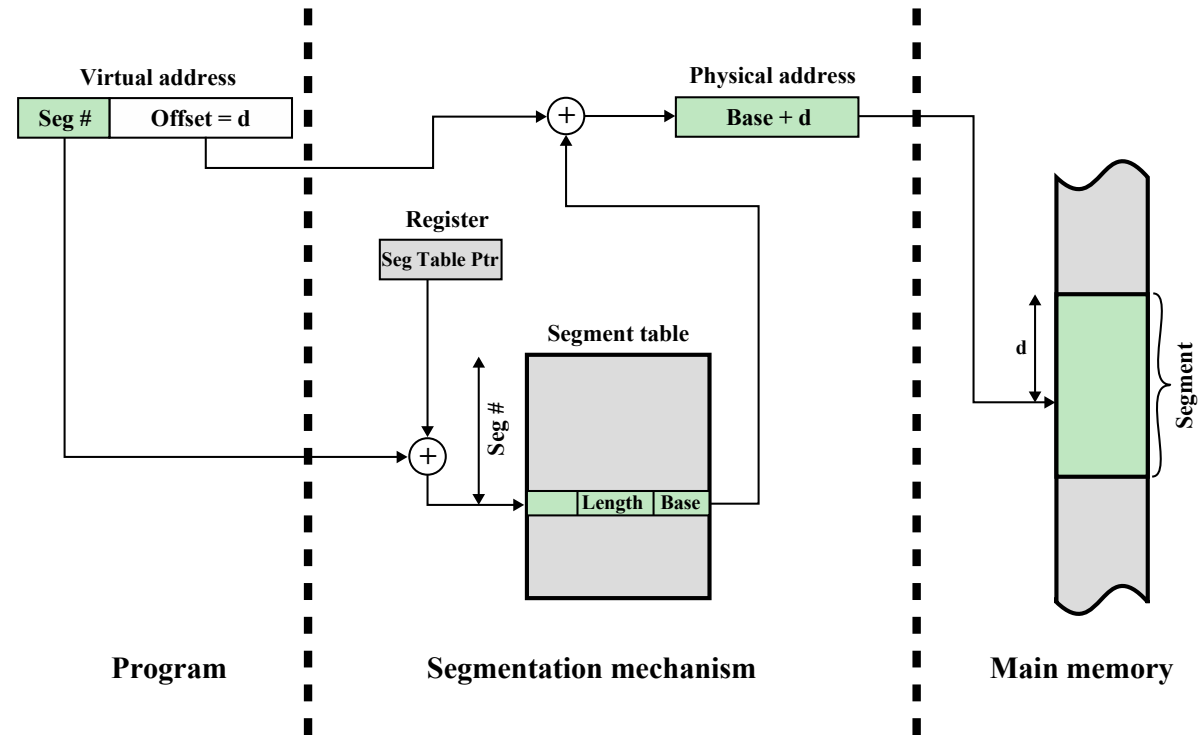


Figure 8.11 Address Translation in a Segmentation System

Paging + Segmentation

In a combined paging/segmentation system a user's address space is broken up into a number of segments. Each segment is broken up into a number of fixed-sized pages which are equal in length to a main memory frame

Segmentation is visible to the programmer

Paging is transparent to the programmer

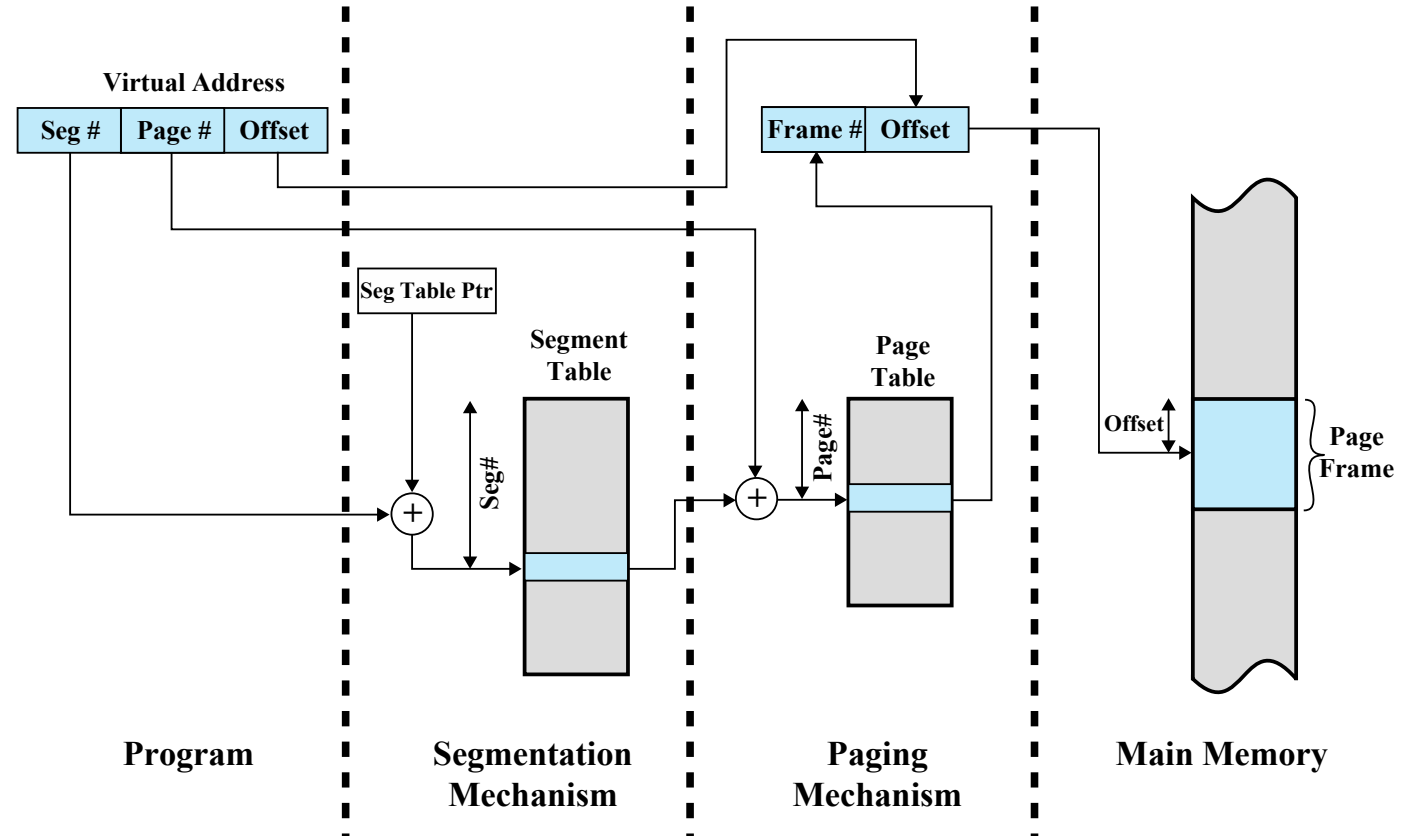


Figure 8.12 Address Translation in a Segmentation/Paging System

Virtual Address



Segment Table Entry



Page Table Entry



P= present bit
M = Modified bit

(c) Combined segmentation and paging

Operating system software

- The design of the memory management portion of an operating system depends on three fundamental areas of choice:
 1. Whether or not to use virtual memory techniques
 2. The use of paging or segmentation or both
 3. The algorithms employed for various aspects of memory management

Fetch Policy

Demand paging

Prepaging

Placement Policy**Replacement Policy**

Basic Algorithms

Optimal

Least recently used (LRU)

First-in-first-out (FIFO)

Clock

Page Buffering

Resident Set Management

Resident set size

Fixed

Variable

Replacement Scope

Global

Local

Cleaning Policy

Demand

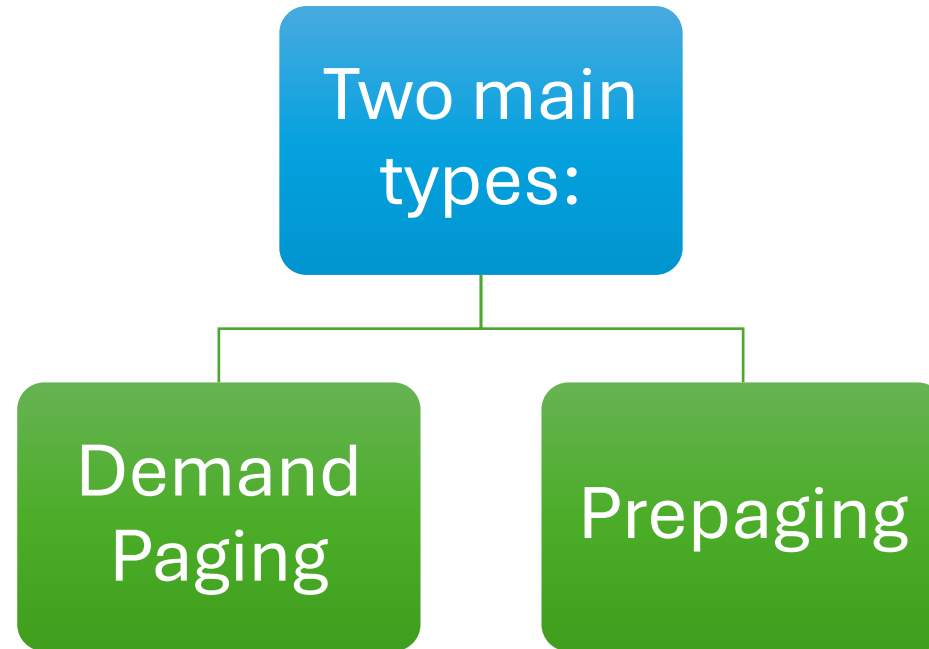
Precleaning

Load Control

Degree of multiprogramming

Fetch Policy

- Determines when a page should be brought into memory



Demand Paging

- Demand Paging
 - Only brings pages into main memory when a reference is made to a location on the page
 - Many page faults when process is first started
 - Principle of locality suggests that as more and more pages are brought in, most future references will be to pages that have recently been brought in, and page faults should drop to a very low level

Prepaging

- **Prepaging**

- Pages other than the one demanded by a page fault are brought in
- Exploits the characteristics of most secondary memory devices
- If pages of a process are stored contiguously in secondary memory it is more efficient to bring in a number of pages at one time
- Ineffective if extra pages are not referenced
- Should not be confused with “swapping”

Placement Policy

- Determines where in real memory a process piece is to reside
- Important design issue in a segmentation system
- Paging or combined paging with segmentation placing is irrelevant because hardware performs functions with equal efficiency.

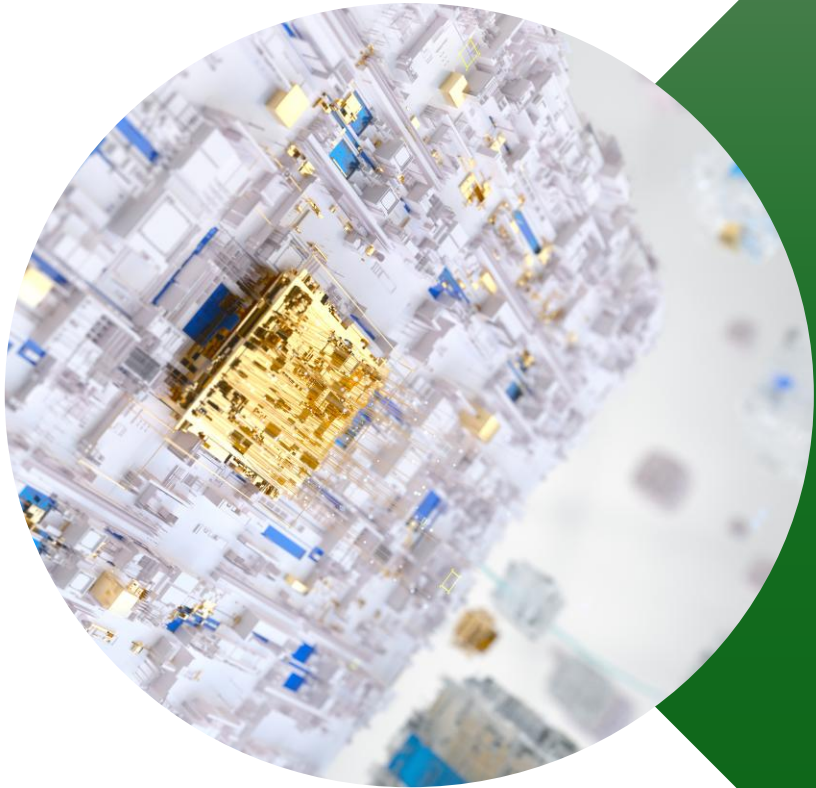
Replacement Policy

- Deals with the selection of a page in main memory to be replaced when a new page must be brought in
 - Objective is that the page that is removed be the page least likely to be referenced in the near future
- The more elaborate the replacement policy the greater the hardware and software overhead to implement it

Frame Locking

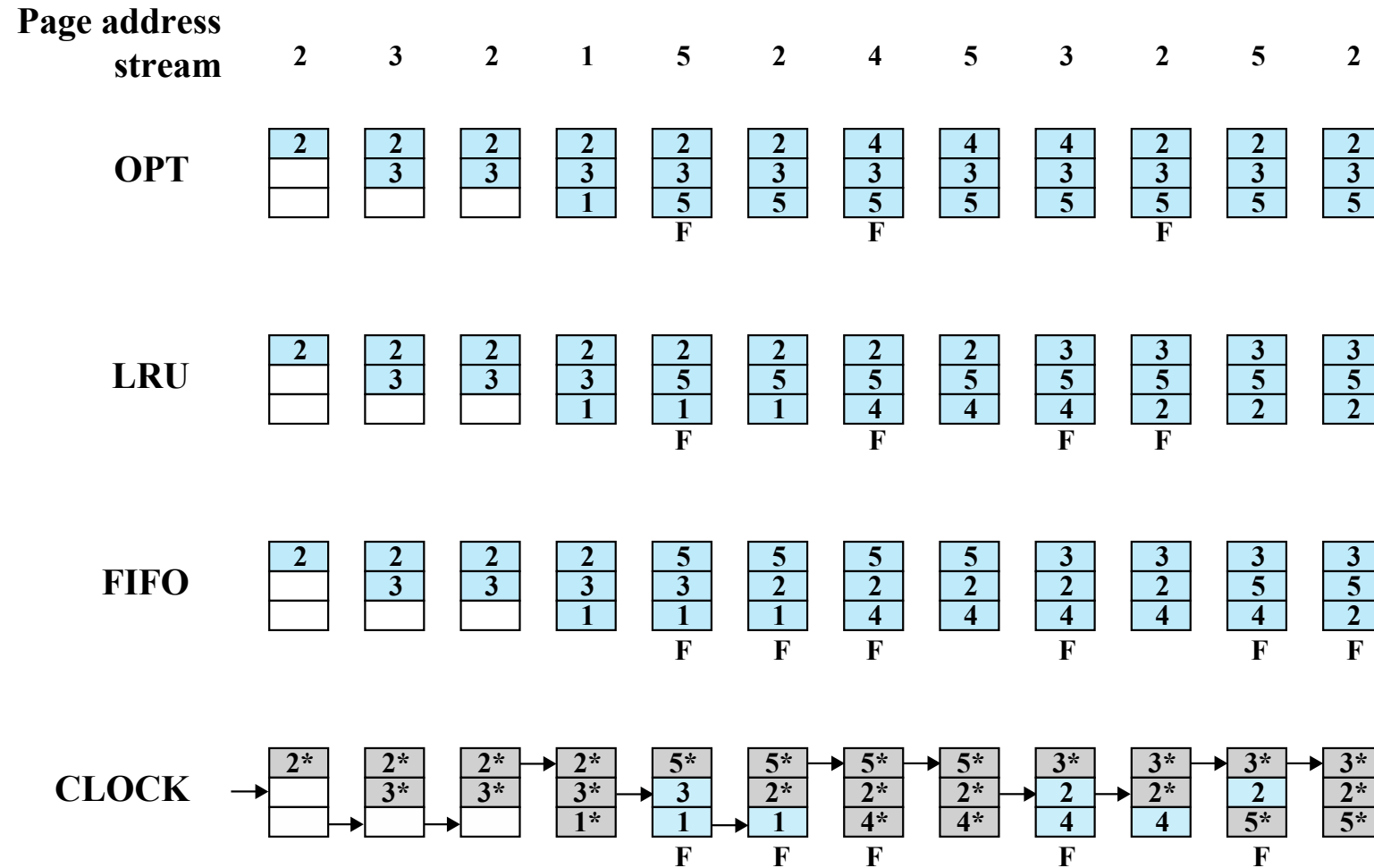
- When a frame is locked the page currently stored in that frame may not be replaced
 - Kernel of the OS as well as key control structures are held in locked frames
 - I/O buffers and time-critical areas may be locked into main memory frames
 - Locking is achieved by associating a lock bit with each frame

Basic Algorithms



Algorithms used for the selection of a page to replace:

- Optimal
- Least recently used (LRU)
- First-in-first-out (FIFO)
- Clock



F = page fault occurring after the frame allocation is initially filled

Figure 8.14 Behavior of Four Page-Replacement Algorithms

Animation



page_replacement (2).html

Resident Set Management

- The OS must decide how many pages to bring into main memory
 - The smaller the amount of memory allocated to each process, the more processes can reside in memory
 - Small number of pages loaded increases page faults
 - Beyond a certain size, further allocations of pages will not affect the page fault rate

Resident Set Size

- Fixed Allocation
 - Gives a process a fixed number of frames in main memory within which to execute
 - When a page fault occurs, one of the pages of that process must be replaced
- Variable Allocation
 - Allows the number of page frames allocated to a process to be varied over the lifetime of the process
 - Appears to be more powerful.
 - Requires the operating system to assess the behavior of active processes.

Replacement Scope

- The scope of a replacement strategy can be categorized as global or local
 - Both types are activated by a page fault when there are no free page frames

Local

- Chooses only among the resident pages of the process that generated the page fault

Global

- Considers all unlocked pages in main memory

Why paging won against segmentation?

1. Simpler memory management
2. Better support for virtual memory
 - as segments have variable sizes, making swapping and memory protection trickier.
3. Improved memory protection.
 - While segments can have access controls, the variable sizes and logical boundaries make protection less granular and harder to enforce consistently, especially in complex programs.
4. Compatibility with modern hardware
 - Paging aligns better with the design of modern hardware, making it faster and more efficient to implement.